

mobiTrace

Feuille de route

Protection post-quantique

Référence	TRACE-SEC-PQC-2026
Version	1.0
Date	08/08/2026
Auteur	Équipe technique mobiTrace / DGFIP

Résumé exécutif

Ce document définit la stratégie de migration de mobiTrace vers une cryptographie résistante aux ordinateurs quantiques, conformément aux standards NIST FIPS 203/204/205 publiés en août 2024.

La menace principale est le paradigme « Harvest Now, Decrypt Later » (HNDL) : un adversaire peut capturer le trafic TLS chiffré aujourd'hui et le déchiffrer ultérieurement grâce à un ordinateur quantique. La migration est structurée en quatre phases progressives, de la mise à niveau classique immédiate (ECDSA P-384, HS512) jusqu'à l'infrastructure PQC complète (ML-KEM, ML-DSA).

⚠ Ce document contient des informations relatives à la sécurité de l'infrastructure. Ne pas diffuser hors des équipes habilitées.

1. Contexte et menace quantique

1.1 La menace « Harvest Now, Decrypt Later »

Un ordinateur quantique suffisamment puissant (« cryptographiquement pertinent », CRQC) serait capable de casser les algorithmes asymétriques classiques (RSA, ECDSA, ECDH) grâce à l'algorithme de Shor, en temps polynomial. Les algorithmes symétriques (AES, HMAC-SHA) sont affectés par l'algorithme de Grover, qui divise leur niveau de sécurité effectif par deux.

La menace immédiate n'est pas l'existence d'un tel ordinateur — les estimations sérieuses le situent entre 2030 et 2040 — mais la collecte de trafic chiffré aujourd'hui en vue d'un déchiffrement différé. Pour des données devant rester confidentielles sur 10 ans ou plus, la migration doit commencer maintenant.

1.2 Standards NIST (août 2024)

Le NIST a finalisé trois standards de cryptographie post-quantique :

Standard	Algorithme	Type	Usage
FIPS 203	ML-KEM (CRYSTALS-Kyber)	KEM hybride	Echange de clés — remplace RSA/ECDH
FIPS 204	ML-DSA (CRYSTALS-Dilithium)	Signature	Signatures — remplace RSA/ECDSA
FIPS 205	SLH-DSA (SPHINCS+)	Signature	Signatures hash-based, conservateur

Approche recommandée : hybride classique + PQC. Combiner X25519 (classique éprouvé) avec ML-KEM-768 (post-quantique) garantit la sécurité même si l'un des deux algorithmes est ultérieurement compromis.

1.3 Inventaire cryptographique de mobiTrace

Composant	Algorithme actuel	Fichier source	Vulnérabilité QC
Certificat TLS	RSA 2048	postinst:114	CRITIQUE — Shor
Echange clés TLS	RSA/DHE (défaut)	nginx (défaut)	CRITIQUE — Shor
Protocole TLS	TLS 1.2+1.3	nginx	Moyenne
Signature JWT	HS256 / HMAC-SHA256	schema.sql:49	Faible-Moy. — Grover
Secret JWT	48 chars (~285 b)	postinst:17	Faible
Hachage MDP	Bcrypt (bf/8)	schema.sql:100	Faible — symétrique
Transit NFS	Non chiffré	fstab (client)	Moyenne — écoute réseau

2. Feuille de route — Les quatre phases

PHASE 1 — Durcissement classique renforcé — Mois 0 à 2

Effort : ★☆☆☆☆ Impact : ★★☆☆☆

Gains immédiats applicables sans changer de stack technique ni risquer de régression de compatibilité. Ces modifications constituent le socle minimum à déployer en priorité.

1.1 Certificat TLS : RSA 2048 → ECDSA P-384

ECDSA P-384 offre ~192 bits de sécurité classique, soit ~96 bits de sécurité quantique (Grover), contre ~64 bits pour RSA 2048. La modification porte sur une commande dans postinst:114 et est transparente pour tous les navigateurs modernes.

```
# Dans DEBIAN/postinst -- remplacer le bloc openssl existant
openssl ecparam -name secp384r1 -genkey -noout -out "$SSL_DIR/trace.key"
openssl req -new -x509 -key "$SSL_DIR/trace.key" -out "$SSL_DIR/trace.crt" \
-days 825 -sha384 \
-subj "/C=FR/ST=Paris/L=Paris/O=DGFIP/OU=TRACE/CN=localhost"
```

1.2 Nginx : TLS 1.3 exclusif + courbes modernes + headers de sécurité

Forcer TLS 1.3 élimine les suites de chiffrement obsolètes (RC4, 3DES, CBC). L'ajout du header HSTS protège contre les downgrades HTTP. CSP limite la surface XSS.

```
# nginx/sites-available/trace -- dans le bloc server 443
ssl_protocols TLSv1.3;
ssl_prefer_server_ciphers off;
ssl_ecdh_curve X25519:secp384r1;

add_header Strict-Transport-Security "max-age=63072000; includeSubDomains; preload" always;
add_header X-Frame-Options "DENY" always;
add_header X-Content-Type-Options "nosniff" always;
add_header Content-Security-Policy "default-src 'self'; script-src 'self';
style-src 'self' 'unsafe-inline';" always;
```

1.3 JWT : HS256 → HS512 + secret 64 caractères

HS512 avec clé 512 bits offre 256 bits de sécurité quantique effective (après Grover). NIST considère 128 bits quantiques suffisants à horizon 2035 ; 256 bits offre une marge de 20 ans supplémentaires. Modification en deux lignes dans schema.sql, PostgREST supporte HS512 nativement.

```
-- schema.sql:49 et :51 -- remplacer HS256/sha256 par HS512/sha512
header_b64 := auth.url_encode(convert_to('{ "alg": "HS512", "typ": "JWT" }', 'utf8'));
signature := auth.url_encode(hmac(..., convert_to(secret, 'utf8'), 'sha512'));

-- postinst:17 -- passer a 64 caracteres (512 bits d'entropie)
JWT_SECRET=$(tr -dc 'A-Za-z0-9_' < /dev/urandom | head -c 64)
```

Impact utilisateurs : nul. Les sessions actives seront invalides après redémarrage (comportement identique à toute rotation de secret JWT). Une seule reconnexion requise.

PHASE 2 — Hybride PQC pour l'échange de clés TLS — Mois 3 à 9

Effort : ★★☆☆ Impact : ★★★★

Phase la plus structurante : elle neutralise la menace HNDL sur le trafic TLS en activant ML-KEM-768 (FIPS 203) pour l'échange de clés. Nécessite OpenSSL 3.5+, disponible sur Debian 13 (Trixie), cible de déploiement du projet.

2.1 Prérequis : OpenSSL 3.5 sur Debian 13 (Trixie)

OpenSSL 3.5 (avril 2025) intègre ML-KEM nativement sans dépendance externe. Nginx compilé contre OpenSSL 3.5 supporte le groupe X25519MLKEM768.

```
# Verification post-migration Debian 13
openssl version                # doit afficher 3.5.x
openssl list -kem-algorithms | grep ML-KEM  # ML-KEM-512 / 768 / 1024
```

2.2 Nginx : activer le groupe hybride X25519MLKEM768

X25519MLKEM768 est le groupe hybride standardisé (IETF draft) : combinaison de X25519 (classique) + ML-KEM-768 (post-quantique FIPS 203). La directive est un simple ajout dans `ssl_ecdh_curve`, sans rupture de compatibilité : les clients sans support PQC négocient automatiquement X25519 (graceful downgrade).

```
# nginx/sites-available/trace -- bloc server 443
ssl_protocols TLSv1.3;
ssl_ecdh_curve X25519MLKEM768:X25519:secp384r1;
# Ordre : PQC hybride d'abord, fallback classique ensuite
```

2.3 Compatibilité clients

Client	Support X25519MLKEM768	Version minimale
Chrome / Chromium	Oui (depuis v124)	Chrome 124+
Firefox	Oui (depuis v127)	Firefox 127+
Safari	En cours (2026)	Safari 18+
curl + OpenSSL 3.5	Oui	curl 8.3+
Navigateurs anciens	Fallback sur X25519	TLS 1.3 requis

2.4 Validation post-déploiement

```
# Verifier la negociation PQC
openssl s_client -connect localhost:443 -groups X25519MLKEM768 2>&1 \
  | grep 'Server Temp Key'
# Resultat attendu : Server Temp Key: X25519MLKEM768, ...

# Depuis un autre poste du reseau
curl -v --curves X25519MLKEM768 https://<IP_SERVEUR>/api/rpc/me 2>&1 \
  | grep -i 'SSL connection\|curve'
```

Cette phase seule suffit à protéger la confidentialité des échanges contre un adversaire disposant d'un ordinateur quantique en 2030-2040. C'est l'objectif prioritaire.

PHASE 3 — Signatures PQC pour les JWTs — Mois 9 à 18

Effort : ★★★☆ Impact : ★★★☆

HS512 est symétrique : le même secret sert à créer et à vérifier les tokens. Une signature asymétrique post-quantique (ML-DSA) permet de ne jamais exposer la clé privée, même à PostgREST. Phase optionnelle si le secret JWT est suffisamment protégé (accès root uniquement à /etc/trace-api.conf).

3.1 Architecture cible : ML-DSA-65 via plpython3u

PostgreSQL génère et conserve la clé privée ML-DSA dans auth.secrets (inaccessible aux rôles applicatifs). PostgREST reçoit uniquement la clé publique PEM via trace-api.conf.

```
-- Activer Python dans PostgreSQL
CREATE EXTENSION IF NOT EXISTS plpython3u;

-- Generer la paire de cles ML-DSA-65 (une fois, a l'installation)
CREATE OR REPLACE FUNCTION auth.generate_mldsa_keypair()
RETURNS TABLE(private_key text, public_key text) AS $$
import oqs
sig = oqs.Signature('ML-DSA-65')
pub = sig.generate_keypair()
priv = sig.export_secret_key()
return [(priv.hex(), pub.hex())]
$$ LANGUAGE plpython3u;

-- Stockage securise (auth.secrets inaccessible aux roles applicatifs)
INSERT INTO auth.secrets VALUES ('mldsa_private', ...), ('mldsa_public', ...);
```

3.2 Prérequis

- PostgreSQL configuré avec plpython3u (paquet postgresql-plpython3)
- liboqs-python installé : pip3 install liboqs-python
- Vérifier la compatibilité PostgREST avec les JWTs ML-DSA (suivre les releases)

PHASE 4 — Infrastructure PQC complète — Mois 18 à 36

Effort : ★★★★★ Impact : ★★★★★

Cette phase dépend de la maturité de l'écosystème externe (CAs institutionnelles, PostgreSQL, bibliothèques système). Elle finalise la migration vers une surface cryptographique entièrement post-quantique.

Action	Déclencheur	Dépendance externe
Certificats TLS ML-DSA signés CA	Émission FIPS 204 par ANSSI / IGC/A	ANSSI / DigiCert PQ
Argon2id remplace Bcrypt	pgcrypto intègre argon2id	PostgreSQL 18+ / extension
Sauvegardes NFS chiffrées ML-KEM	Avant migration vers stockage cloud	gpg + liboqs ou age-pq
Audit conformité ANSSI / RGS PQ	Avant déploiement production étendue	Prestataire habilité

3. Tableau de bord de synthèse

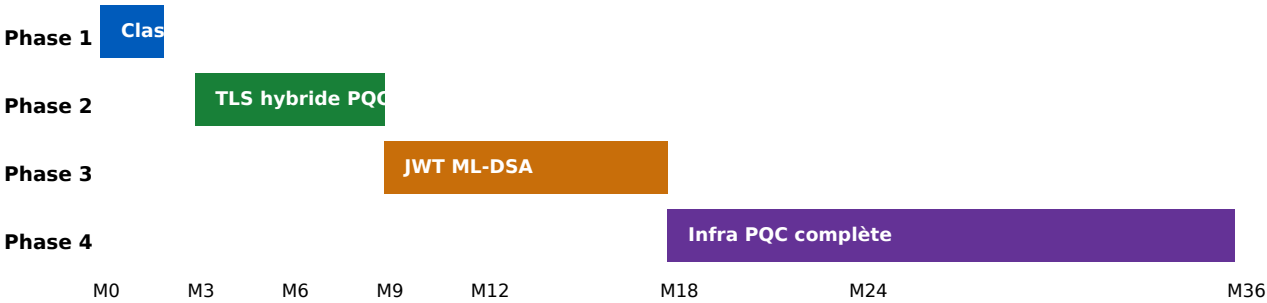
Phase	Horizon	Action clé	Effort	Impact	Statut
Phase 1	0-2 mois	ECDSA P-384 TLS 1.3 + HSTS JWT HS512	☆☆☆☆	★★★★	A planifier
Phase 2	3-9 mois	OpenSSL 3.5 + X25519MLKEM768 (Debian 13)	☆☆☆☆	★★★★	Sous condition
Phase 3	9-18 mois	ML-DSA-65 pour signatures JWT	★★★★	★★★★	Optionnel
Phase 4	18-36 mois	Certificats CA PQC + Argon2id + NFS chiffré	★★★★	★★★★	Ecosystème

Priorités immédiates (Phase 1)

Les trois actions suivantes sont réalisables en moins d'une journée de travail :

1. Générer un certificat ECDSA P-384 dans le script postinst (1 commande openssl)
2. Forcer TLS 1.3 + activer HSTS et CSP dans la configuration Nginx (5 lignes)
3. Passer le JWT de HS256/sha256 à HS512/sha512 dans schema.sql (2 lignes) et étendre le secret à 64 chars dans postinst (1 ligne)

Visualisation temporelle



Références

- NIST FIPS 203 — ML-KEM (Module-Lattice-Based Key-Encapsulation Mechanism)
- NIST FIPS 204 — ML-DSA (Module-Lattice-Based Digital Signature Standard)
- NIST FIPS 205 — SLH-DSA (Stateless Hash-Based Digital Signature Standard)
- OpenSSL 3.5 — ML-KEM et ML-DSA natifs (avril 2025)
- ANSSI — Recommandations de sécurité relatives à la cryptographie post-quantique (2024)
- IETF — Hybrid key exchange in TLS 1.3 (draft-ietf-tls-hybrid-design)
- RFC 8446 — The Transport Layer Security (TLS) Protocol Version 1.3
- PostgREST Documentation — JWT Authentication

Document généré automatiquement par generate_pq_roadmap_pdf.py (dépôt mobiTrace). Pour toute question : équipe technique DGFIP.